

labelled *Reset To Setup Mode* or something similar. Activate it. While you're at it, look for an option labelled *Restore Factory Keys* or something similar. If it's there, we don't need to back up our keys. If it isn't, we do.

Now that we're in *Setup Mode*, reboot the system and boot from the USB Key. *KeyTool* should launch automatically.

KeyTool is menu-driven and very simple to use. Begin by backing up your existing keys if you want to. You'll get three files—*PK.esl*, *KEK.esl*, and *db.esl*, which should be saved to the root of your USB drive. These are all ESL files, and can be concatenated with your own ESL files (with the exception of the PK, because there can only be one PK) if you need to import the existing keys into your system.

Now begin enrolling your keys. Do it in this order—db, KEK and PK—choosing the *Replace Keys* option for every one of them. Enrol *db.auth* for the db database, *KEK.auth* for the KEK database and *PK.auth* for the PK database.

The moment you enrol the PK, the system is switched to *User Mode*. You're now ready to boot binaries signed by any of the db keys enrolled in your system. Cross your fingers and reboot. If you have done this correctly, you should be able to boot straight to your desktop. If not, well, get into your firmware, reset to *Setup Mode* and try again.

Continued from page No. 74...

would mean that if your server encounters a URL like `http://servername/JSP/something`, the request will be forwarded to the Tomcat servers based upon the *workers.properties* file. This directive holds comma separated values. In our case, it is `/*` which means to forward all requests to the application servers.

Having done everything else, let's move towards the last step of this configuration, i.e., creating the *workers.properties* file inside the `/etc/httpd/conf.d` directory as follows:

```
worker.list = LoadBalancerCluster

worker.tomcatOne.type = ajp13
worker.tomcatOne.host = 192.168.1.253
worker.tomcatOne.port = 8009
worker.tomcatOne.lbfactor = 1
worker.tomcatOne.connection_pool_size = 64
worker.tomcatOne.connection_pool_minsize = 16
worker.tomcatOne.connection_pool_timeout = 120

worker.tomcatTwo.type = ajp13
worker.tomcatTwo.host = 192.168.1.254
worker.tomcatTwo.port = 8009
worker.tomcatTwo.lbfactor = 1
worker.tomcatTwo.connection_pool_size = 64
worker.tomcatTwo.connection_pool_minsize = 16
```

Final thoughts

Is UEFI worth it? Definitely. It's faster, programmatically cleaner, and if done right, easier to manage than GRUB, Syslinux or whatever your favourite bootloader is. The UEFI shell is the icing on the cake. It's simple, powerful and scriptable.

Is Secure Boot worth it? No. It's a royal pain in the posterior. If you're using a distribution that supports Secure Boot (i.e., Fedora and/or Ubuntu), then the whole signing business is handled for you automatically and you have to do none of this. If, however, you choose to use a DIY distro (like Arch or Gentoo), you're going to have to sign every kernel update manually. Moreover, there's no real point to Secure Boot on Linux unless the *initramfs* and kernel modules are signed too, and the file system is encrypted, decryption of which is somehow related to your Secure Boot certificates. Simply booting a signed binary, which then ends up loading a ton of arbitrary unsigned modules two milliseconds later, is a waste. 

By: Boudhayan Gupta

Describing himself as a 'retard by choice', the author believes that madness is a cure-all for whatever is wrong or right with society. A social media enthusiast, he can be reached at @BaloneyGeek at Twitter.

```
worker.tomcatTwo.connection_pool_timeout = 120
```

```
worker.LoadBalancerCluster.type = lb
worker.LoadBalancerCluster.sticky_session = 1
worker.LoadBalancerCluster.sticky_session_force = 1
worker.LoadBalancerCluster.balance_workers = tomcatOne,
tomcatTwo
```

The configuration part is now complete. It is worth mentioning that the value of *worker.list* in the first line of the file is the same as the string after *JkMount* `/*` in *mod-jk.conf*. All you need to do now is to start the two Tomcat instances one after the other, using their default *startup.sh* file, start the *httpd* server and test your configuration.

Upon completion, the above configuration will result in a set-up in which the end users will not notice any difference or performance downtime even if the application server with which their session was created crashes during the session. The user will continue to access your site as if nothing has happened. 

By: Abhinav Rai

The author is an open source enthusiast and a networking blogger. He has been working on technologies like Linux, Java/J2EE, Ruby/Ruby on Rails and MySQL for the last three years. His major interest lies in deploying various network services on Linux servers.